Java Silver（SE 17・1Z0-825-JPN）　4 クラスの定義とインスタンス　#09

static とアクセス修飾子

クラスに 1 つのものと、オブジェクトごとのもの。どこから見えるかを決める 4 つのレベル

各ページの下に、根拠にした Oracle の資料（JLS 6・8・12・15 章、Java Tutorials）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 4（static、初期化子、アクセス修飾子、カプセル化と不変オブジェクト）

1 static フィールド
クラスに 1 つ。全インスタンスで共有

2 static メソッド
オブジェクトなしで呼ぶ。this とインスタンスメンバーは直接使えない

3 static 初期化子
クラスの初期化で 1 回だけ。上から順

4 アクセス修飾子
public・protected・なし・private

5 カプセル化と不変
private で隠し、変えられないオブジェクトを作る

	static あり	static なし
フィールド	クラス変数（1 つ）	インスタンス変数（オブジェクトごと）
メソッド	クラスメソッド	インスタンスメソッド
初期化子	static { } クラスで 1 回	{ } new のたび（#08）
呼び方	クラス名.名前	変数.名前
this	使えない	使える

static とアクセス修飾子は別の軸。private static も public static もある。static だから広く使えるわけではない。

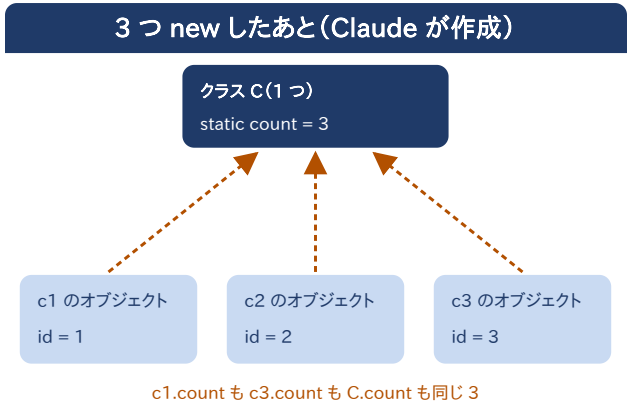
出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／The Java Tutorials「Understanding Class Members」

第1章

static フィールド

どの変数から触っても、同じ 1 つ。

クラス変数は全インスタンスで 1 つ




```
class C {
  static int count;     // クラスに 1 つ
  int id;               // オブジェクトごと
  C() { count++; id = count; }
}
// new C() を 3 回 → c1, c2, c3
System.out.println(C.count + " " + c1.id + " " + c3.id + " " + c1.count);
```

```
3 1 3 3
```

コード	結果（JDK 17）
a.s = 5; のあと b.s・C.s	5・5
a.n = 5; のあと b.n	0（別の変数）
static final int X;（代入なし）	variable X not initialized in the default constructor

出典：JLS 8.3.1.1 static Fields（there exists exactly one incarnation of the field, no matter how many instances (possibly zero) of the class may eventually be created）・8.3.1.2（A blank final class variable must be definitely assigned by a static initializer）／Java Tutorials「Understanding Class Members」（Every instance of the class shares a class variable）／結果は JDK 17 で確かめたもの

第2章

static メソッド

今のオブジェクトがない場所。this もインスタンス変数も、そのままでは使えない。

static コンテキストで使えるもの

どこから	何を	JDK 17
static メソッド	static のメンバー	OK
	インスタンス変数 n	non-static variable n cannot be referenced from a static context
	this.n	non-static variable this …
	new A().k	OK（オブジェクトを通す）
インスタンスメソッド	static もインスタンスも	OK

理由：static メソッドは特定のオブジェクトなしで呼ばれるので、「どのオブジェクトの n か」が決まらない。

main からインスタンスメソッドを呼ぶ（Claude が作成）

```
public class Main {
  void hi() { System.out.println("hi"); }
  public static void main(String[] args) {
    hi();              // コンパイルエラー
    new Main().hi();    // OK → hi
  }
}
```

```
error: non-static method hi() cannot be referenced from a static context
```

static コンテキスト：static メソッド・static 初期化子・static フィールドの初期化式の中。this と super は使えない。

出典：JLS 8.4.3.2 static Methods（Notably, the keywords this and super are prohibited in a static context, as are unqualified references to instance variables, instance methods …）／Java Tutorials（Class methods cannot access instance variables or instance methods directly—they must use an object reference）／結果は JDK 17 で確かめたもの

null の変数でも static は使える

```
class C {
  static int s = 7;
  static String hello() { return "hello"; }
  int n = 1;
}
C c = null;
System.out.println(c.s + " " + c.hello());   // 7 hello
System.out.println(c.n);                     // NullPointerException
```

static は宣言された型で決まる。c.s は C.s と同じ意味で、c の中身（オブジェクト）は使わない。だから null でも例外にならない。

コード（c は null の C 型変数）	結果（JDK 17）
c.s（static フィールド）	7
c.hello()（static メソッド）	hello
c.n（インスタンス変数）	NullPointerException

変数を通して static を使う書き方は、読む人に誤解を与えるのですすめられない。試験では「例外になる」をえらばせるひっかけに使われる。

count の動き（Claude が作成）	count
C.next(); C.next();	2
new C().count += 10;	12（new で 0 に戻らない）
C.next()	13

出典：JLS 15.11.1（If the field is static: The Primary expression is evaluated, and the result is discarded）・15.12.4.1（If the invocation mode is static, then there is no target reference）／結果は JDK 17 で確かめたもの

第3章

static 初期化子

クラスがはじめて使われる直前に 1 回だけ。

初期化のタイミングと順番

```
class A {
  static { System.out.print("s1 "); }
  { System.out.print("i "); }
  A() { System.out.print("c "); }
  static { System.out.print("s2 "); }
}
// main：print("m "); new A(); new A();
```

```
m s1 s2 i c i c
```



こんなとき	クラスは初期化される？
main のあるクラス	main の前に（static 初期化子が先）
`A a = null;`（宣言だけ）	されない
はじめての `new A()`	される
はじめての `A.f()`	される
2 回目の `A.f()`	もう済んでいる

```
A a = null;  print("1 ");  A.f();  A.f();
// 1 A-init f f
```

出典：JLS 8.7 Static Initializers（A static initializer declared in a class is executed when the class is initialized）／JLS 12.4.1 When Initialization Occurs（T is a class and an instance of T is created. A static method declared by T is invoked …）／結果は JDK 17 で確かめたもの

上から順 ― 前方参照に注意

```
class A {
  static int x;            // 0
  static { x = 5; }          // 5
  static int y = x * 2;    // 10
}
System.out.println(A.x + " " + A.y);   // 5 10
```

static フィールドの初期化式と static 初期化子は、ひとまとめのブロックのように上から順に実行される。static 初期化子が全部のあとにまとめて動くわけではない。

コード	結果（JDK 17）
`static int a = b + 1;` `static int b = 2;`	illegal forward reference
`static int a = A.b + 1;` `static int b = 2;`	a は 1、b は 2
`static final int X;` `static { X = 3; }`	OK（3）

A.b と書くと前方参照のチェックを通るが、その時点の b はまだ既定値 0。だから a は 1 になる（試験ではまれ。しくみの理解用）。

出典：JLS 12.4.2（execute either the class variable initializers and static initializers of the class … in textual order, as though they were a single block）／JLS 8.3.3（For a reference by simple name to a class variable f … at a point to the left of f's declarator）／Java Tutorials「Initializing Fields」／結果は JDK 17 で確かめたもの

第4章

アクセス修飾子

同じクラス・同じパッケージ・サブクラス・どこからでも。

4 つのアクセスレベル

修飾子	同じクラス	同じパッケージ	別パッケージのサブクラス	どこでも
public	○	○	○	○
protected	○	○	○	×
なし	○	○	×	×
private	○	×	×	×

広い順：public → protected → なし（パッケージアクセス）→ private。protected は「なし」の範囲に、別パッケージのサブクラスを足したもの。

「なし」を default と呼ぶこともあるが、default という修飾子を書くわけではない。

エラーの表示（JDK 17）	
使ったもの	メッセージ
private（別クラスから）	n has private access in A
なし（別パッケージから）	def is not public in A; cannot be accessed from outside package
protected（別パッケージの非サブクラスから）	pro has protected access in A

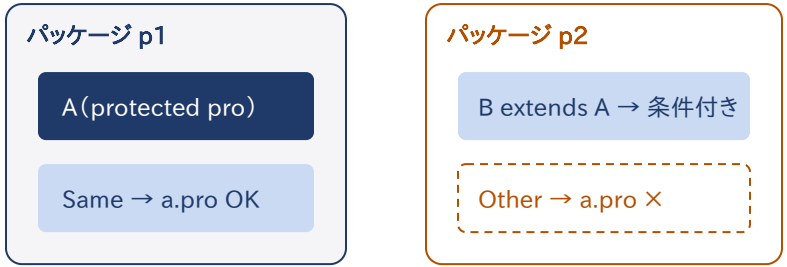
private はクラス単位。同じクラスの中なら、別のインスタンスの private フィールド（o.n）も使える。同じファイルの別クラスからは使えない。

出典：Java Tutorials「Controlling Access to Members of a Class」（Access Levels の表／The protected modifier specifies that the member can only be accessed within its own package … and, in addition, by a subclass of its class in another package）／JLS 6.6.1／結果は JDK 17 で確かめたもの（パッケージは p1・p2 にファイルを分けた）

protected のくわしいルール

p1 の A と、p2 の B（A のサブクラス）（Claude が作成）

```
package p1;
public class A {
  protected int pro = 2;
  protected static int ps = 5;
}
// ---- 別ファイル ----
package p2;
public class B extends p1.A {
  void t(p1.A a, B b) {
    int v1 = pro;        // OK
    int v2 = this.pro;   // OK
    int v3 = b.pro;      // OK（B 型）
    int v4 = a.pro;      // エラー（A 型）
    int v5 = ps;         // OK
  }
}
```



別パッケージのサブクラスでの条件：インスタンスのフィールド・メソッドは、自分（B）か B のサブクラスの型を通したときだけ。親の型 A の変数を通すとエラー。

サブクラスでない p2 のクラスからは、protected は static（A.ps）でも使えない。使えるのは public だけ。

出典：JLS 6.6.2.1 Access to a protected Member（Access is permitted only within the body of a subclass S of C … access to the instance field Id is permitted if and only if the qualifying type is S or a subclass of S）／結果は JDK 17 で確かめたもの

修飾子を付けられる場所

書いたもの	結果（JDK 17）
`private class A { }`（トップレベル）	modifier private not allowed here
`protected class A { }`（トップレベル）	modifier protected not allowed here
`class A { }`（トップレベル）	OK（パッケージアクセス）
`public private int x;`	illegal combination of modifiers: public and private
main の中で `private int x = 0;`	illegal start of expression
main の中で `static int x = 0;`	illegal start of expression
main の中で `final int x = 0;`	OK

場所	付けられるアクセス修飾子
トップレベルクラス	public か なし
フィールド・メソッド・コンストラクタ	4 つのどれか 1 つ
クラスの中のクラス	4 つのどれか 1 つ
ローカル変数	なし（final だけ付けられる）

> **public のトップレベルクラスは 1 ファイルに 1 つ**で、ファイル名と同じ名前（#01）。

出典：JLS 8.1.1 Class Modifiers（The access modifiers protected and private pertain only to member classes）・8.3.1（more than one of the access modifiers public, protected, and private）／JLS 14.4（VariableModifier: Annotation | final）／Java Tutorials（At the top level—public, or package-private）／結果は JDK 17 で確かめたもの

第5章

カプセル化と不変オブジェクト

隠して、決めた入口からだけ触らせる。

カプセル化と private コンストラクタ

```
class Account {
  private int balance;              // 隠す
  void deposit(int x) {
    if (x > 0) balance += x;          // 入口で確かめる
  }
  int getBalance() { return balance; }
}
a.deposit(100);  a.deposit(-50);
System.out.println(a.getBalance());   // 100
```

カプセル化: フィールドを private にし、メソッドを通してだけ読み書きさせる。おかしな値をメソッドではじける。

Java Tutorials:「意味のある範囲でいちばん制限の強いアクセスレベルを使う。理由がなければ private」。

```
class S {
  private S() { }
  static S create() { return new S(); }   // 中からは OK
}
S s1 = new S();      // エラー
S s2 = S.create();    // OK
```

```
error: S() has private access in S
```

private コンストラクタ: 外から new させず、作り方を static メソッド(ファクトリメソッド)に限る。

出典:Java Tutorials「Controlling Access to Members of a Class」(Use the most restrictive access level that makes sense for a particular member. Use private unless you have a good reason not to)／JLS 6.6.1／結果は JDK 17 で確かめたもの

不変オブジェクトの作り方

1 setter を作らない

2 フィールドはすべて private final

3 クラスを final にする（またはコンストラクタを private にしてファクトリメソッド）

4 配列などはコピーを保存し、コピーを返す

Java Tutorials「A Strategy for Defining Immutable Objects」の手順を Claude がまとめた。

final の参照フィールド list	結果（JDK 17）
`list.add("x")`	OK（中身は変えられる）
`list = new ArrayList<>()`	cannot assign a value to final variable list

コピーしないと外から変えられる（Claude が作成）

```
final class P {
  private final int[] data;
  P(int[] d) { data = d; }          // ×そのまま
  int[] get() { return data; }      // ×そのまま
}
int[] src = {1, 2};
P p = new P(src);
src[0] = 9;  p.get()[1] = 9;
```

P の書き方	p.get() の中身
そのまま保存・そのまま返す	9 9（変えられた）
data = d.clone();・return data.clone();	1 2（守られた）

出典：Java Tutorials「A Strategy for Defining Immutable Objects」（Don't provide "setter" methods／Make all fields final and private／Never store references to external, mutable objects passed to the constructor; if necessary, create copies）／JLS 4.12.4（the state of the object may be changed … but the variable will always refer to the same object）／結果は JDK 17 で確かめたもの

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
static フィールドの表示	1 つしかない。どの変数を通して変えても同じもの。new で 0 に戻らない
static メソッドがコンパイルできるか	this・インスタンス変数・インスタンスメソッドを名前だけで使っていないか。オブジェクトを通せば OK
null の変数で static を使う	例外にならない。宣言された型のメンバーが使われる
初期化の表示順	クラスの初期化（static、1 回）→ new のたびにインスタンス初期化子 → コンストラクタ。宣言だけでは初期化されない
アクセスできるか	表で考える。protected は別パッケージのサブクラスで自分の型を通したときだけ。private はクラス単位
修飾子の付け方	トップレベルは public かなし。ローカル変数は final だけ。アクセス修飾子は 1 つ
不変クラス	private final、setter なし、final クラス、コピーを保存・返す。final は中身を守らない

出典：このPDFの各ページの資料

まとめ

static フィールド
クラスに 1 つ。全インスタンスで共有

static メソッド
this なし。インスタンスはオブジェクト経由で

static 初期化子
クラスの初期化で 1 回。上から順

アクセス修飾子
public・protected・なし・private

カプセル化と不変
private で隠し、コピーで守る

こんなとき	結果
main から同じクラスの hi()（static なし）	コンパイルエラー
null の C 型変数で c.s（static）	値が読める
`A a = null;` だけ	A は初期化されない
トップレベルクラスに private	コンパイルエラー
final の配列フィールドの要素を変える	変えられる

次は #10 パターンマッチングとレコード。instanceof のパターン、record のしくみと自動で作られるメンバーへ進む。

出典：このPDFの各ページの資料